CY TECH

FONDAMENTAUX DU DEEP LEARNING • ING3

SignaVision

Reconnaissance de panneaux de signalisation par réseau convolutif



Nathan GUIHOT **Elyes DOS SANTOS**

Encadrante : Hanane Zerdoum • 16 octobre 2026

Le problème

Une photo de panneau en entrée, sa catégorie en sortie



Préparation
32×32×3
pixels dans [0, 1]

CNN
convolutions
ReLU, pooling

Softmax
43 probabilités
de somme 1



Stop
100,0 %

Classification multiclasse. 43 classes, donc 43 neurones de sortie et une softmax : $p_k = e^{z_k} / \sum_j e^{z_j}$. On prédit la classe la plus probable. Avec deux classes, un neurone et une sigmoïde auraient suffi.

Démarche, en six étapes

1 données **2** réseau dense **3** CNN
4 entraînement **5** analyse des erreurs
6 expériences et modèle final

GTSRB : 51 839 photos réelles de 43 types de panneaux

German Traffic Sign Recognition Benchmark (Stallkamp et al., IJCNN 2011)

39 209
images d'entraînement

12 630
images de test

43
classes, en 6 familles

43 px
largeur et hauteur médianes



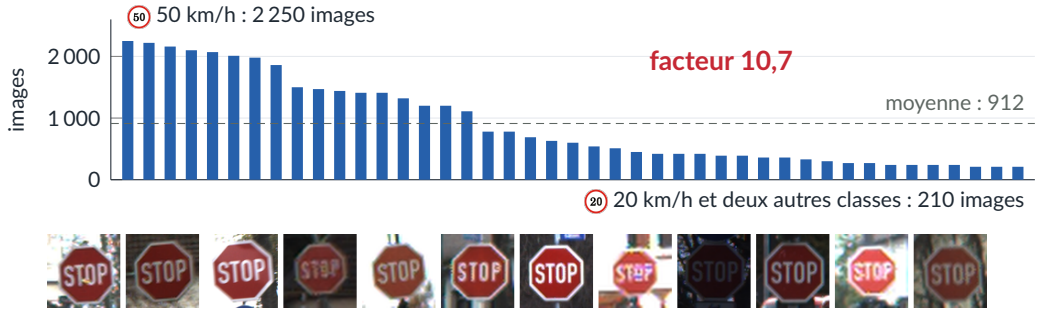
Une image d'entraînement par classe, de 0 à 42.

PNG couleur, marge d'environ 10 % autour du panneau, **séquences de 30 photos** du même panneau.

Source : benchmark.ini.rub.de (copie Kaggle). Libre pour la recherche et l'enseignement, avec citation.

Ce qui rend la tâche difficile

Des classes déséquilibrées et des photos très variables



Douze panneaux Stop issus de séquences différentes : sombres, flous, surexposés, minuscules (17 % des images ont un côté sous 32 px).

Préparation des données

Tout passe par la même fonction `preprocess`

Redimensionnement en 32×32. Entrée de taille fixe ; assez pour lire un panneau, modèle léger.

Normalisation. Pixels divisés par 255 : entrées dans $[0, 1]$, gradients plus stables.

Labels. Entiers de 0 à 42 et perte *sparse categorical cross-entropy* : même calcul qu'avec des labels one-hot, sans stocker la matrice.



Une ligne = une séquence de 30 photos quasi identiques

entraînement validation

Train **31 379** · Validation **7 830** · Test **12 630**

Validation : 20 % des séquences de chaque classe.
Le test officiel n'est ouvert qu'à la toute fin.

Un réseau dense comme point de comparaison

Image → Flatten → Dense(256, ReLU) → Dense(43, softmax)



797 739 paramètres

Propagation avant : $a^{(l)} = f\big(W^{(l)} a^{(l-1)} + b^{(l)}\big)$, poids W et biais b appris.
Activation : ReLU $\max(0, z)$, sans quoi le réseau resterait linéaire.
Perte : entropie croisée, $L = -\log p_y$.
Rétropropagation : $\partial L / \partial w$ par dérivation en chaîne.
Descente de gradient : $w \leftarrow w - \eta\, \partial L / \partial w$ (Adam).

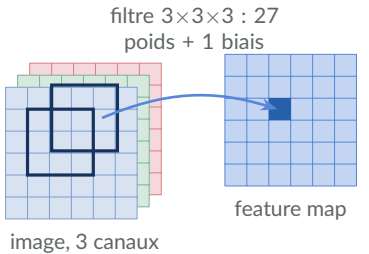
83,14 %
accuracy de validation

91,34 %
accuracy d'entraînement

Chaque pixel est une entrée indépendante : décaler le panneau de deux pixels change toute l'entrée. Le réseau mémorise plus qu'il ne reconnaît des formes.

Ce qu'apporte la convolution

Des filtres locaux, partagés sur toute l'image



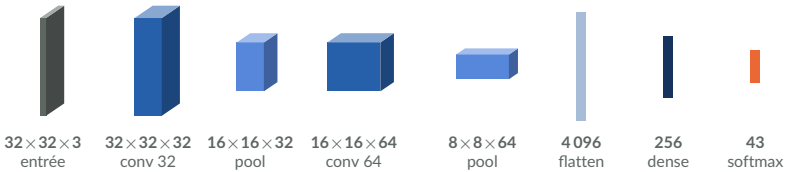


Un Stop et 8 des 32 feature maps de la 1re couche du CNN v1.

- **Filtre** : détecte un motif local (bord, couleur, chiffre), où qu'il soit.
- **Feature maps** : une carte par filtre, empilées comme des canaux.
- **Kernel 3×3, stride 1** : deux 3×3 empilées couvrent une zone 5×5 avec 18 poids au lieu de 25 par canal.
- **Padding** `same` : taille conservée. Sortie : $\lfloor (n + 2p - k)/s \rfloor + 1$, ici $(32 + 2 - 3)/1 + 1 = 32$.
- **ReLU** $\max(0, z)$ après chaque convolution.
- **Max pooling 2×2** : hauteur et largeur divisées par 2, moins sensible aux petits décalages.

Notre CNN de départ (v1)

Deux blocs convolution + ReLU + max pooling, puis une couche dense



Couche	Sortie	Paramètres
Conv2D 32, 3×3	32×32×32	$(3 \cdot 3 \cdot 3 + 1) \cdot 32 = 896$
Conv2D 64, 3×3	16×16×64	$(3 \cdot 3 \cdot 32 + 1) \cdot 64 = 18\,496$
Dense 256	256	$4\,096 \cdot 256 + 256 = 1\,048\,832$
Dense 43	43	$256 \cdot 43 + 43 = 11\,051$
Total		**1 079 275**

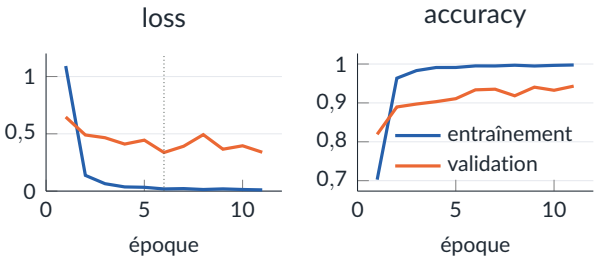
Filtres doublés (32 puis 64) quand la résolution est divisée par 2. Base volontairement simple, que la partie 6 fait évoluer.

Moins de 20 000 paramètres dans les convolutions, grâce au partage des poids. **97 % des paramètres** sont dans la couche dense après le Flatten : c'est là que le modèle peut surapprendre.

Entraînement du CNN v1

Réglages communs au réseau dense, au CNN v1 et aux expériences

Perte : entropie croisée
Optimiseur : Adam, $\eta = 10^{-3}$
Batch : 64 (environ 490 mises à jour par époque)
Époques : 30 au plus, arrêt anticipé, patience 5
Métriques : accuracy, et F1 macro (moyenne des F1 par classe) à cause du déséquilibre



Surapprentissage net : 99,69 % sur le train, 93,36 % en validation. La perte de validation est minimale à l'époque 6 puis oscille ; l'arrêt anticipé garde ces poids (pointillés).

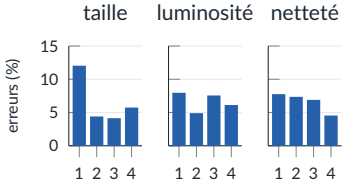
Où se trompe le CNN v1 ?

Validation : 520 erreurs sur 7 830 images

Confusions les plus fréquentes

Vraie classe	Prédite	Nb	%
Virage à gauche	Chaussée glissante	30	100,0 %
Chaussée rétrécie	Feux tricolores	23	38,3 %
Carrefour giratoire	Contournement à droite	22	36,7 %
Limitation 30 km/h	Limitation 50 km/h	19	4,2 %
Passage piétons	Feux tricolores	13	21,7 %
Limitation 30 km/h	Limitation 80 km/h	11	2,4 %

Taux d'erreur par quartile



Quartile 1 : les plus petites, les plus sombres, les plus floues.

Même forme, même couleur, seul le pictogramme change : toute la séquence « virage à gauche » est prédite « chaussée glissante ».

Images ≤ 34 px : **2 à 3 fois plus d'erreurs**. Classes rares plus dures ($\rho = 0{,}50$).

Cinq expériences, une modification à la fois

Comparaison sur la validation, à partir du CNN v1

	Modification	Accuracy de validation	F1 macro	Écart train/val
v1	référence (2 blocs, 1 conv. par bloc)	93,36 %	0,879	6,3 pts
E1	3e bloc (128 filtres)	92,15 %	0,857	7,4 pts
E2	2 convolutions par bloc	97,10 %	0,956	2,9 pts
E3	dropout (0,25 puis 0,5)	96,48 %	0,932	3,4 pts
E4	batch normalization	94,32 %	0,901	5,6 pts
E5	data augmentation	93,50 %	0,893	6,3 pts



Augmentation : rotation $\pm 10°$, zoom, décalage, luminosité, contraste. Pas de retournement : « tourner à gauche » deviendrait « tourner à droite ».

E2 et E3 : les seuls gains nets (+3 à 4 points).

E1 et E4 restent dans le bruit, mais E1 a 42 % de paramètres en moins.

E5 apprend lentement : l'arrêt anticipé l'a sans doute coupé trop tôt.

E6 : le piège de la validation aléatoire

Même CNN v1, seule la façon de tirer la validation change

99,53 %
validation tirée image par image

93,36 %
validation tirée par séquence

94,98 %
vraie accuracy sur le test (mesurée à la fin)

Image par image, presque chaque image de validation a une quasi-copie dans le train (même séquence). Le score gagne des points qui n'existent pas : 99,53 % affichés, 94,98 % en réalité.

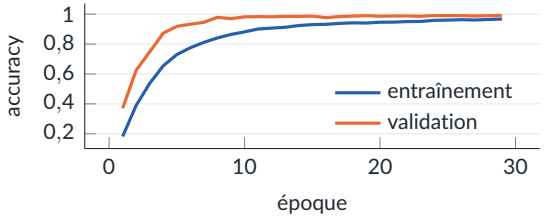
Par séquence, la validation prédit bien le test :

CNN v1	93,36 % →	92,00 %
modèle final	98,95 % →	98,35 %

Le modèle final

Les cinq modifications, combinées et entraînées plus longtemps

3 blocs de **2 convolutions** 3×3 (32, 64, 128 filtres)
Batch normalization et **dropout**
Data augmentation
60 époques au plus, patience 10, learning rate divisé par 2 en cas de plateau



Plus de surapprentissage : écart train/val de 0,9 pts (6,3 pts pour le v1). La courbe d'entraînement est sous la validation car mesurée avec augmentation et dropout actifs.

98,95 %
accuracy de validation

823 947
paramètres

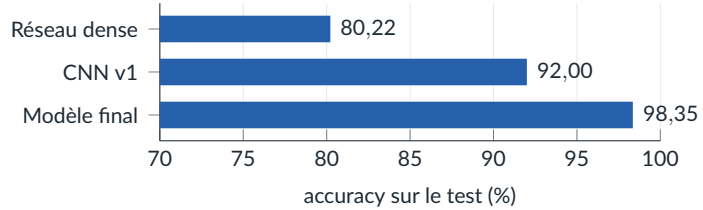
Résultats sur le jeu de test

12 630 images jamais vues, évaluées une seule fois, à la fin

98,35 %
accuracy du modèle final

0,964
F1 macro

209
erreurs sur 12 630



Limitations	99,4 %	Danger	94,4 %	Priorité	99,4 %
Interdictions	99,7 %	Obligations	99,2 %	Fins d'interdiction	94,7 %

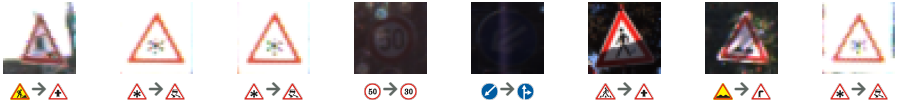
Accuracy du modèle final par famille de panneaux.

Les erreurs qui restent

Modèle final, jeu de test

Vraie classe	Prédite	Nb	%
Succession de virages	Priorité à l'intersection	30	33,3 %
Passage piétons	Priorité à l'intersection	29	48,3 %
Neige ou verglas	Chaussée glissante	24	16,0 %
Fin d'interdiction (> 3,5 t)	Fin d'interdiction de dépasser	13	14,4 %
Autres dangers	Succession de virages	8	2,1 %

Ces confusions font **presque la moitié** des 209 erreurs. Elles arrivent **par paquets** : même scène, même erreur, souvent avec une forte confiance.

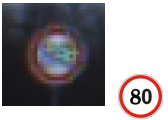


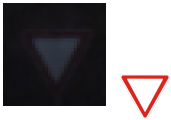
Les 4 erreurs commises avec la plus forte probabilité, puis 4 au hasard : vraie classe → classe prédite, probabilité.

Démonstration du modèle final

`predict_sign(image)` : même `preprocess`, puis les 3 classes les plus probables







En direct. On appelle `predict_sign("photo.jpg")` sur une photo hors dataset, recadrée sur le panneau comme les images GTSRB.

Pictogrammes officiels : 26 sur 43. Formes et couleurs reconnues, mais les limitations dessinées sont presque toutes lues « 30 » : le modèle a appris les photos, pas le panneau idéal.

Ce qu'on retient

Résultats

Accuracy sur les 12 630 images du test :

réseau dense	**80,22 %**
CNN v1	**92,00 %**
modèle final	**98,35 %**

F1 macro du modèle final : 0,964.

Leçons

Valider par séquence : tirée image par image, la validation surestime l'accuracy de 4,5 points.

Deux convolutions par bloc et le dropout : les deux vrais gains.

Erreurs restantes : panneaux de même forme, quelques séquences atypiques, classes rares.

Limites et suites

Un seul entraînement par configuration : il faudrait plusieurs graines pour départager les écarts de 1 point.

En 32×32, les petits pictogrammes deviennent illisibles.

Suites possibles : transfer learning, détection YOLO dans une scène complète, vidéo.

Merci

Des questions ?



Nathan GUIHOT Elyes DOS SANTOS • CY Tech, ING3